Snickr — Project Report (Part1)

Xingyu Li
xl5936@nyu.edu

Yishu Xu
yx3935@nyu.edu

(a) Database Schema Design

System Overview

Users register with an email address, choose a username and password, and then participate in one or more *workspaces*. Within each workspace, members communicate through *channels*, each of which holds a chronologically ordered sequence of *messages*. Access control is layered: workspace membership is required before a user can see or join any channel inside that workspace, and channel membership is required before a user can read or post messages.

The schema is organised around four functional modules:

Module	**Core tables**
User management	`users`
Workspace management	`workspaces`, `roles`, `workspacemember`, `workspaceinvitation`
Channel management	`channeltype`, `channels`, `channelmember`, `channelinvitation`
Message management	`messages`

Three small lookup tables (`roles`, `status`, `channeltype`) replace hard-coded string literals, keeping the schema open to adding new values without a structural change.

ER Diagram

The full entity–relationship diagram is stored at `docs/ER-Diagram.drawio.svg` and was drawn with draw.io. It covers all eleven tables, their attributes, and the cardinalities of every relationship.

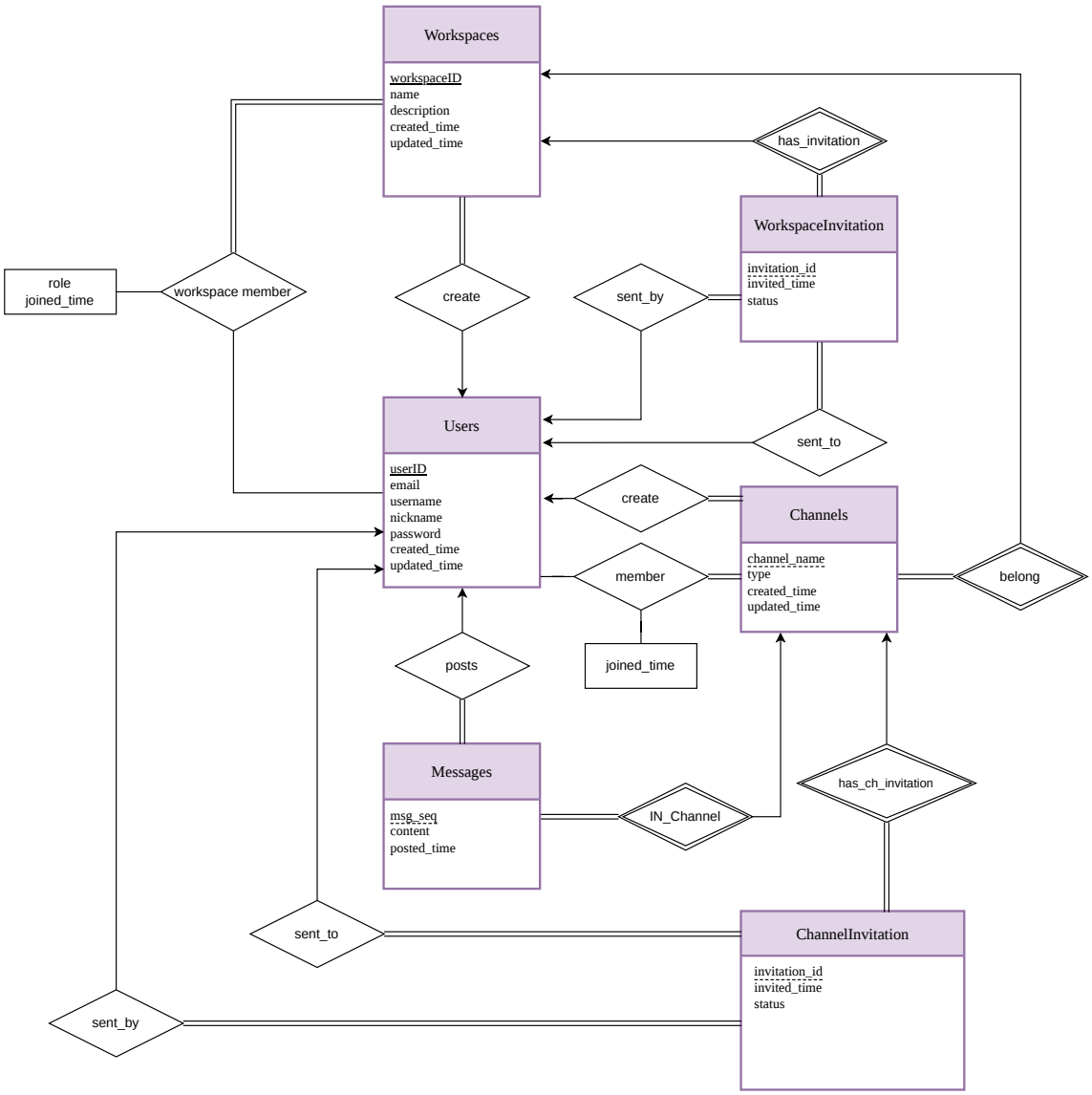

Figure 1: Entity–Relationship diagram of the Snickr database.

The four primary entities are:

- **User** — identified by `userID`; holds authentication and profile data.
- **Workspace** — identified by `workspaceID`; the top-level grouping unit.
- **Channel** — identified by `channelID`; belongs to exactly one workspace.
- **Message** — identified by `messageID`; belongs to exactly one channel.

The major relationships and their cardinalities are:

Relationship	Cardinality	Realised by
User *creates* Workspace	1 : N	`workspaces.created_by`
User *belongs to* Workspace	M : N	`workspacemember`
User *invites* User to Workspace	M : N	`workspaceinvitation`
Workspace *contains* Channel	1 : N	`channels.workspaceID`
User *creates* Channel	1 : N	`channels.created_by`
User *belongs to* Channel	M : N	`channelmember`
User *invites* User to Channel	M : N	`channelinvitation`
User *posts* Message	1 : N	`messages.posted_by`
Channel *contains* Message	1 : N	`messages.channelID`

Relational Schema

The translation from the ER diagram yields the following eleven relations. Primary keys are **bold**; constraints are listed in the rightmost column.

`users`

Column	Type	Constraints
`userID`	INTEGER	PK, identity
`email`	VARCHAR(50)	UNIQUE, NOT NULL
`username`	VARCHAR(30)	UNIQUE, NOT NULL
`nickname`	VARCHAR(30)	—
`password`	VARCHAR(30)	—
`created_time`	TIMESTAMP	NOT NULL, default NOW()
`updated_time`	TIMESTAMP	NOT NULL, default NOW()

`workspaces`

Column	Type	Constraints
`workspaceID`	INTEGER	PK, identity
`name`	VARCHAR(30)	NOT NULL
`description`	VARCHAR(200)	—
`created_time`	TIMESTAMP	NOT NULL, default NOW()
`updated_time`	TIMESTAMP	NOT NULL, default NOW()
`created_by`	INTEGER	nullable, FK → `users.userID` ON DELETE SET NULL

`created_by` is intentionally nullable. Because the delete policy is `SET NULL`, the column must accept NULL so that deleting a user does not cascade-delete all workspaces they founded.

`roles`

Column	Type	Constraints
`roleID`	INTEGER	PK, identity
`name`	VARCHAR(20)	UNIQUE, NOT NULL

Seeded values: `'admin'`, `'member'`.

`workspacemember`

Column	Type	Constraints
`workspaceID`	INTEGER	PK (composite), FK → `workspaces.workspaceID` ON DELETE CASCADE
`userID`	INTEGER	PK (composite), FK → `users.userID` ON DELETE CASCADE
`role`	INTEGER	NOT NULL, FK → `roles.roleID`
`joined_time`	TIMESTAMP	NOT NULL, default NOW()

The composite PK `(workspaceID, userID)` enforces that a user can appear at most once in any given workspace.

`status`

Column	Type	Constraints
`statusID`	INTEGER	PK, identity
`type`	VARCHAR(30)	UNIQUE, NOT NULL

Seeded values: `'pending'`, `'accepted'`, `'declined'`. Shared by both `workspaceinvitation` and `channelinvitation` so the invitation lifecycle is defined in one place.

`workspaceinvitation`

Column	Type	Constraints
`invitationID`	INTEGER	PK, identity
`workspaceID`	INTEGER	NOT NULL, FK → `workspaces.workspaceID` ON DELETE CASCADE
`invitee`	INTEGER	NOT NULL, FK → `users.userID` ON DELETE CASCADE
`inviter`	INTEGER	NOT NULL, FK → `users.userID`
`invited_time`	TIMESTAMP	NOT NULL, default NOW()
`status_type`	INTEGER	NOT NULL, FK → `status.statusID`

Unique constraint on `(workspaceID, invitee)` prevents duplicate invitations to the same workspace for the same user.

`channeltype`

Column	Type	Constraints
`typeID`	INTEGER	PK, identity
`name`	VARCHAR(30)	UNIQUE, NOT NULL

Seeded values: `'public'`, `'private'`, `'direct'`.

`channels`

Column	Type	Constraints
`channelID`	INTEGER	PK, identity
`workspaceID`	INTEGER	NOT NULL, FK → `workspaces.workspaceID` ON DELETE CASCADE
`channel_name`	VARCHAR(50)	NOT NULL
`typeID`	INTEGER	NOT NULL, FK → `channeltype.typeID`
`created_by`	INTEGER	NOT NULL, FK → `users.userID`
`created_time`	TIMESTAMP	NOT NULL, default NOW()
`updated_time`	TIMESTAMP	NOT NULL, default NOW()

Unique constraint on `(workspaceID, channel_name)` enforces that channel names are unique within a workspace while allowing the same name to exist in different workspaces (matching Slack's behaviour).

`channelmember`

Column	Type	Constraints
`channelID`	INTEGER	PK (composite), FK → `channels.channelID` ON DELETE CASCADE
`userID`	INTEGER	PK (composite), FK → `users.userID` ON DELETE CASCADE
`joined_time`	TIMESTAMP	NOT NULL, default NOW()

Composite PK `(channelID, userID)` prevents duplicate memberships.

`channelinvitation`

Column	Type	Constraints
`invitationID`	INTEGER	PK, identity
`channelID`	INTEGER	NOT NULL, FK → `channels.channelID` ON DELETE CASCADE
`invitee`	INTEGER	NOT NULL, FK → `users.userID` ON DELETE CASCADE
`inviter`	INTEGER	NOT NULL, FK → `users.userID`
`invited_time`	TIMESTAMP	NOT NULL, default NOW()
`status_type`	INTEGER	NOT NULL, FK → `status.statusID`

Unique constraint on `(channelID, invitee)` mirrors the workspace invitation constraint.

`messages`

Column	Type	Constraints
`messageID`	INTEGER	PK, identity
`channelID`	INTEGER	NOT NULL, FK → `channels.channelID` ON DELETE CASCADE
`content`	VARCHAR(500)	NOT NULL
`posted_time`	TIMESTAMP	NOT NULL, default NOW()
`posted_by`	INTEGER	NOT NULL, FK → `users.userID`

Design Assumptions and Justifications

Workspace creator becomes admin automatically. The `created_by` column records who founded the workspace. On creation, the application inserts a corresponding row into `workspacemember` with `role = 'admin'`. This is confirmed by the sample data, where alice (`created_by` of Acme Corp) and bob (`created_by` of Math Club) both appear in `workspacemember` with the admin role.

Any workspace member can create a channel. The project specification says only to "check that the user is authorized". The design interprets authorization as workspace membership: any member, regardless of role, may create a channel. This is enforced in query (c.2) via a `WHERE EXISTS` check against `workspacemember`. Restricting channel creation to admins only would limit collaboration without a clear stated requirement to do so.

Role granularity is workspace-level only. Channels do not have their own role system. Channel access is controlled entirely through `channeltype` (public / private / direct) and the invitation flow. This avoids a combinatorial explosion of role assignments across potentially many channels per workspace.

`status` is shared by both invitation tables. Workspace and channel invitations share the same lifecycle (`pending` → `accepted / declined`). A single `status` lookup table is referenced by both. If a new state (e.g., `expired`) is ever needed, it is added in one place.

Channel name uniqueness is scoped to the workspace. The unique constraint is `(workspaceID, channel_name)`, not globally on `channel_name`. This matches Slack's behaviour and user expectations: a `#general` channel in one company's workspace is unrelated to `#general` in another's.

Timestamps on every mutable entity. `created_time` and `updated_time` are stored on users, workspaces, and channels. `posted_time` is stored on messages; `joined_time` on membership rows; `invited_time` on invitation rows. This makes queries like "invited more than 5 days ago" (c.4) answerable purely from stored data.

No threading, no message deletion, no payment tiers. Per the project specification, threading within a channel is not required. Messages are append-only; there is no `deleted_at` column. Payment and tier information are out of scope.

(b) Schema Implementation

The full DDL is in `database/schema/schema.sql`. This section describes the constraint strategy and index design.

Identity Columns

Every synthetic primary key uses `INTEGER GENERATED ALWAYS AS IDENTITY`, the SQL-standard equivalent of `SERIAL`. The database exclusively controls these values. New IDs are retrieved with `RETURNING` on insert, avoiding a second round-trip.

Constraints

Primary keys. Single-column PKs are defined on the identity column of every main table. Composite PKs are defined on the join tables `workspacemember (workspaceID, userID)` and `channelmember (channelID, userID)`.

Foreign keys and delete behaviour.

Relationship	Policy	Rationale
`workspaces.created_by` → `users`	SET NULL	Workspace survives creator deletion; `created_by` must be nullable
`workspacemember` → `users`, `workspaces`	CASCADE	No orphan memberships
`workspaceinvitation.invitee` → `users`; `workspaceinvitation` → `workspaces`	CASCADE	Invitations are meaningless without the invitee or workspace
`workspaceinvitation.inviter` → `users`	no action	Historical record; inviter may be deleted later
`channels` → `workspaces`	CASCADE	A channel cannot exist without its workspace
`channels.created_by` → `users`	no action	Channel record is preserved even if creator is deleted
`channelmember` → `users`, `channels`	CASCADE	No orphan memberships
`channelinvitation.invitee` → `users`; `channelinvitation` → `channels`	CASCADE	Same rationale as workspace invitations
`channelinvitation.inviter` → `users`	no action	Historical record
`messages` → `channels`	CASCADE	Messages belong entirely to their channel
`messages.posted_by` → `users`	no action	Message content is preserved even if the poster is deleted

Unique constraints. `users.email` and `users.username` are globally unique. `(workspaceID, invitee)` on `workspaceinvitation` and `(channelID, invitee)` on `channelinvitation` prevent duplicate invitations. `(workspaceID, channel_name)` on `channels` enforces scoped channel name uniqueness.

NOT NULL. Every column that the system requires for correct operation is marked NOT NULL. Optional profile fields (`nickname`, `description`) and the workspace `created_by` audit column are left nullable intentionally.

Defaults. All timestamp columns default to `CURRENT_TIMESTAMP`. The application only needs to supply explicit values when overriding the recording time, which is never necessary in normal operation.

Index Design

Three additional indexes are created beyond the implicit PK indexes:

Index	Table	Column	Purpose
`idx_messages_channel`	`messages`	`channelID`	Fetch all messages for a channel — the most frequent query
`idx_workspace_member_user`	`workspacemember`	`userID`	Find all workspaces a user belongs to
`idx_channel_member_user`	`channelmember`	`userID`	Find all channels a user belongs to

(c) The Seven Queries

Parameterised versions of all queries (using `:placeholder` syntax) are in `database/queries/queries.sql`. The versions below with concrete values substituted are in `database/seeds/test_queries.sql`.

(c.1) Register a new user

Requirement: insert a new user given an email, username, nickname, and password.

```sql
INSERT INTO users (email, username, nickname, password,
                   created_time, updated_time)
VALUES (:email, :username, :nickname, :password, NOW(), NOW())
RETURNING userID;
```

The `userID` column is `GENERATED ALWAYS AS IDENTITY`, so the new ID is assigned by the database. Returning it lets the application know which row was just created without an extra round trip. Both timestamps are set to `NOW()` so that the audit fields match the moment of registration.

(c.2) Create a public channel inside a workspace

Requirement: a particular user creates a new public channel inside a workspace, but only if that user is currently a member of the workspace.

```sql
BEGIN;

WITH new_channel AS (
    INSERT INTO channels (workspaceID, channel_name, typeID, created_by,
                          created_time, updated_time)
    SELECT :workspace_id,
           :channel_name,
           (SELECT typeID FROM channeltype WHERE name = 'public'),
           :user_id,
           NOW(), NOW()
    WHERE EXISTS (
        SELECT 1 FROM workspacemember
        WHERE workspaceID = :workspace_id AND userID = :user_id
    )
    RETURNING channelID
)
INSERT INTO channelmember (channelID, userID, joined_time)
SELECT channelID, :user_id, NOW() FROM new_channel;
```

```sql

COMMIT;
```

The authorisation check is the `WHERE EXISTS` inside the `INSERT ...SELECT`. If the requesting user is not in the target workspace, the subquery returns no row and the channel is never inserted. Because the second `INSERT` reads from the same CTE, it also inserts nothing. The two statements are wrapped in a transaction so the channel and its initial membership row commit together.

The `typeID` for `'public'` is looked up by name rather than hard-coded so that the query is independent of the actual identity values in `channeltype`.

(c.3) List all administrators of every workspace

Requirement: for each workspace, list its current administrators.

```sql
SELECT  w.workspaceID, w.name AS workspace_name,
        u.userID, u.username, u.nickname
FROM    workspaces      w
JOIN    workspacemember wm ON wm.workspaceID = w.workspaceID
JOIN    roles           r  ON r.roleID       = wm.role
JOIN    users           u  ON u.userID       = wm.userID
WHERE   r.name = 'admin'
ORDER BY w.workspaceID, u.username;
```

The role is stored as a foreign key into the `roles` lookup table, so the filter joins through `roles` and matches by name. Sorting by `workspaceID` first keeps administrators of the same workspace grouped together.

(c.4) Pending invitations older than five days

Requirement: for each public channel in a given workspace, count the number of users who were invited more than five days ago and have still not joined.

```sql
SELECT  c.channelID, c.channel_name,
        COUNT(ci.invitationID) AS pending_invites_over_5_days
FROM    channels    c
JOIN    channeltype ct ON ct.typeID = c.typeID
LEFT JOIN channelinvitation ci
       ON ci.channelID    = c.channelID
      AND ci.invited_time < NOW() - INTERVAL '5 days'
      AND ci.status_type  = (SELECT statusID FROM status WHERE type =
          'pending')
      AND NOT EXISTS (
            SELECT 1 FROM channelmember cm
            WHERE cm.channelID = ci.channelID AND cm.userID = ci.invitee
      )
WHERE   c.workspaceID = :workspace_id
  AND   ct.name       = 'public'
GROUP BY c.channelID, c.channel_name
ORDER BY c.channel_name;
```

Two design choices are worth noting:

- The age filter, status filter, and "has not yet joined" check are all in the `LEFT JOIN`'s `ON` clause rather than in `WHERE`. This ensures a public channel with zero qualifying invitations still appears in the result with a count of zero, rather than being dropped.
- The "has not yet joined" check uses `NOT EXISTS` against `channelmember` rather than relying solely on the `status` column. An invitation whose status is still `pending` while the user has joined through another path would be incorrectly counted if the status alone were used.

(c.5) All messages in a particular channel

Requirement: list all messages of a given channel in chronological order.

```sql
SELECT  m.messageID, m.content, m.posted_time,
        u.userID, u.username, u.nickname
FROM    messages m
JOIN    users    u ON u.userID = m.posted_by
WHERE   m.channelID = :channel_id
ORDER BY m.posted_time ASC, m.messageID ASC;
```

The secondary sort on `messageID` is a tiebreaker for the case where two messages share a timestamp; without it the display order would be non-deterministic. The `messages` table has an index on `channelID` (`idx_messages_channel`), which is the access path this query uses.

(c.6) All messages posted by a particular user

Requirement: list all messages posted by a particular user across any channel.

```sql
SELECT  m.messageID,
        w.workspaceID, w.name AS workspace_name,
        c.channelID,   c.channel_name,
        m.content, m.posted_time
FROM    messages   m
JOIN    channels   c ON c.channelID   = m.channelID
JOIN    workspaces w ON w.workspaceID = c.workspaceID
WHERE   m.posted_by = :user_id
ORDER BY m.posted_time DESC, m.messageID DESC;
```

The result includes the workspace and channel name so that the message stream is readable without further lookups. Newest first matches how a typical activity feed is presented. No access filter is applied here because (c.6) concerns the user's own posts, which the user can always see.

(c.7) Keyword search restricted to accessible messages

Requirement: for a particular user, list every message that contains the keyword "perpendicular" in its body, restricted to messages the user is allowed to see (must be a member of both the workspace and the specific channel).

```sql
SELECT  m.messageID,
        w.workspaceID, w.name AS workspace_name,
        c.channelID,   c.channel_name,
        poster.username AS posted_by,
        m.content, m.posted_time
FROM    messages        m
JOIN    channels        c  ON c.channelID    = m.channelID
JOIN    workspaces      w  ON w.workspaceID  = c.workspaceID
JOIN    channelmember   cm ON cm.channelID   = c.channelID   AND cm.userID =
    :user_id
JOIN    workspacemember wm ON wm.workspaceID = w.workspaceID AND wm.userID =
    :user_id
JOIN    users           poster ON poster.userID = m.posted_by
WHERE   m.content ILIKE '%perpendicular%'
ORDER BY m.posted_time ASC, m.messageID ASC;
```

Access control is expressed as two `JOIN`s: one against `channelmember` filtered to the requesting user, and one against `workspacemember` filtered to the same user. A message survives the joins only when the user is a member of both the channel and its enclosing workspace. The workspace check is technically implied by the channel check today (a user cannot be in a channel without being in the workspace it belongs to), but stating both makes the access rule explicit and keeps the query correct if that invariant is ever weakened.

`ILIKE` is used so the search is case-insensitive; `LIKE` would miss `Perpendicular` at the start of a sentence.

(d) Sample Data and Tests

Description of the Test Dataset

The test dataset is small enough to reason about row by row, while still covering the positive and negative branches of every query in part (c). It contains 6 users, 2 workspaces, 5 channels, 4 channel invitations, and 9 messages.

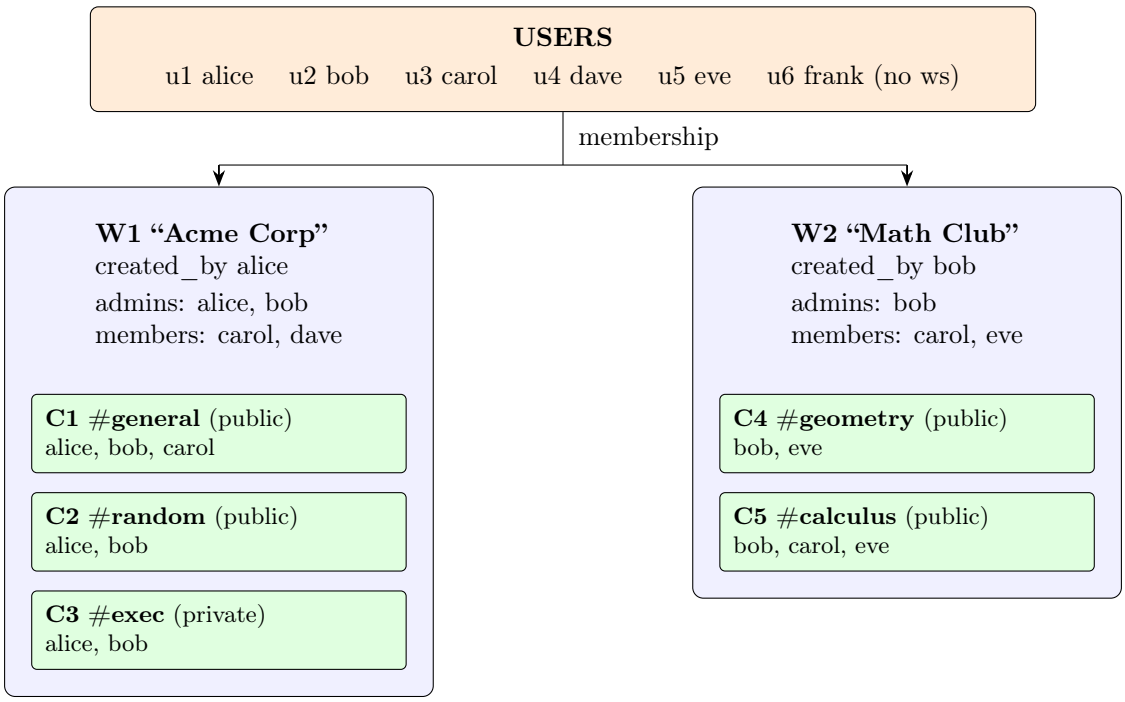

Figure 2: Test dataset overview — 6 users, 2 workspaces, 5 channels.

Two users are deliberately given unusual memberships:

- **frank** (u6) is registered but belongs to no workspace. He is the empty-result case for any query that depends on workspace or channel membership.
- **carol** (u3) belongs to both workspaces but only to a subset of the channels in each. She lets (c.7) include some of her accessible messages and exclude others within the same query run.

Channel invitations are placed in W1 only, since (c.4) takes a single workspace as input. Each invitation is positioned to fall on a different side of the filter:

Channel	Invitee	Age (days)	Status	Joined?	Counted by (c.4)?
C1 #general	dave	7	pending	no	yes
C2 #random	carol	10	pending	no	yes
C2 #random	dave	3	pending	no	no — age $\leq$ 5 days
C3 #exec (priv)	carol	20	pending	no	no — private

Of the nine messages, six contain the keyword "perpendicular". They are spread across access boundaries so that (c.7) returns visibly different result sets for different users:

#	Channel	Poster	Age (days)	Contains "perpendicular"?
1	C1 #general	alice	5	no
2	C1 #general	bob	4	yes
3	C1 #general	carol	3	yes
4	C2 #random	alice	2	yes
5	C3 #exec	alice	1	yes
6	C4 #geometry	bob	10	yes
7	C4 #geometry	eve	9	no
8	C5 #calculus	carol	8	yes
9	C5 #calculus	bob	7	no

The full INSERT script is `database/seeds/sample_data.sql` and is idempotent — it truncates the business tables and reseeds them on every run, so identity sequences always restart from 1 and subsequent test runs remain deterministic.

Test Results

The runnable tests live in `database/seeds/test_queries.sql`. Each query below substitutes concrete IDs from the sample data for the placeholders. Timestamps use the form $T - N$d, where T is the moment `sample_data.sql` was executed.

(c.1) — register `george`

The insert returns one row. The `userID` is 7 because the seed already fills 1–6.

userID	email	username
7	george@acme.com	george

`george` is then deleted to keep the dataset stable for the queries that follow.

(c.2) — create a channel, with and without authorisation

Test A. carol (u3, a member of Acme) creates `#announcements` in W1. The new channel and its initial membership row are both written:

channelID	workspaceID	channel_name	typeID	created_by
6	1	announcements	1	3

channelID	userID	joined_time
6	3	$T - 0$

Test B. frank (u6, not a member of Acme) tries to create `#hack` in the same workspace. The membership check in the `WHERE EXISTS` clause fails, so no row is written:

must_be_zero
0

The two tests together show that the same query performs the work and silently rejects unauthorised attempts — there is no need for a separate permission check at the application layer.

(c.3) — administrators per workspace

workspaceID	workspace_name	userID	username	nickname
1	Acme Corp	1	alice	Ali
1	Acme Corp	2	bob	Bobby
2	Math Club	2	bob	Bobby

Acme Corp returns two administrators (alice and bob); Math Club returns one (bob). bob appears twice because he is an administrator of both workspaces.

(c.4) — old pending invitations in Acme

channelID	channel_name	pending_invites_over_5_days
1	general	1
2	random	1

The private channel `#exec` does not appear because the outer `WHERE` filters to public channels. The recent invitation to dave on `#random` (3 days old) is excluded by the age filter on the join, and the 20-day-old invitation on `#exec` is excluded twice over (private channel and not in the result set).

(c.5) — messages in `#general`

messageID	content	posted_time	username
1	Hello team, welcome to Acme general channel!	$T - 5$d	alice
2	Our new logo features perpendicular lines for a modern look.	$T - 4$d	bob
3	Love the perpendicular motif — very clean.	$T - 3$d	carol

Three messages, oldest first, as expected.

(c.6) — messages posted by alice

messageID	workspace	channel	content	posted_time
5	Acme Corp	exec	Confidential — we need a perpendicular strategy for Q4.	$T - 1$d
4	Acme Corp	random	Off-topic: perpendicular is my favorite geometry word.	$T - 2$d
1	Acme Corp	general	Hello team, welcome to Acme general channel!	$T - 5$d

Alice's three posts come back newest first. They span both public channels (`general`, `random`) and a private one (`exec`); no visibility filter is applied because (c.6) is about the user's own posts.

(c.7) — keyword search visible to carol

messageID	workspace	channel	posted_by	content	posted_time
8	Math Club	calculus	carol	Recall: the tangent and normal vectors are perpendicular.	$T - 8$d
2	Acme Corp	general	bob	Our new logo features perpendicular lines for a modern look.	$T - 4$d
3	Acme Corp	general	carol	Love the perpendicular motif — very clean.	$T - 3$d

Three of the six "perpendicular" messages survive the access filter for carol. The other three are correctly excluded:

Message	Channel	Why carol cannot see it
M4 alice	`#random` (W1)	not a member of `#random`
M5 alice	`#exec` (W1)	not a member of `#exec` (private channel)
M6 bob	`#geometry` (W2)	not a member of `#geometry`

As a sanity check, the same query is run for frank (u6), who has no workspace or channel memberships:

frank_accessible_count
0

Frank cannot reach any channel, and the result is empty as required.

Summary

All seven queries return the expected rows on the sample data. The positive cases (admins listed, messages returned, accessible content matched) and the negative cases (unauthorised inserts, recent invitations, private channels, users with no memberships) all hold. The data intentionally populates every filter boundary, so a later regression — for instance, accidentally treating a private channel as public, or losing the age filter in (c.4) — would change the returned counts and be caught by re-running these tests.